



GETTING STARTED WITH THE KRIA KR260 ROBOTICS STARTER KIT

A SIMPLE EXAMPLE PROJECT

Author : Bevilacqua Mattia

Date : Dec. 2024

Preface

The aim of this guide is to give a reference design flow for the creation of a project using both Vivado and Petalinux SDK. Within this way of programming, the bare metal approach, such as the one normally used for STM micro controllers, gives more space to a OS-oriented method. To all intents and purposes, after the creation of the project, what is left is a tailor-made OS that will manage the board in a similar way to that of the bare metal, but with all the advantages of having a OS.

Even though some passages are explained, it is given for granted that the developer has some basic knowledge of the Vivado and Petalinux environments. While a the creation of the Vivado- Petalinux project and further implementations are illustrated, it is left for the reader to find a way to create the environment needed to follow the steps disclosed in the following. To write this document and to create the project the following software/devices have been used:

- Kria KR260 Robotic Starter Kit
- Vivado 2024.1
- Petalinux SDK v2022.2
- CentOS stream 9

Newest software releases might not be compatible with the procedure(s) listed and/or with the files used to support the processes. It is recommended to check resources availability and compatibility before the installation of Vivado and, paying more attention, of Petalinux SDK.

The guide is divided into four chapters. The first chapter gives an overview of the Petalinux installation procedure with the bespoke OS (Operative System). The second chapter deals with the creation of a simple project that will exploit the programmable logic (PL). This method is called bare-metal programming. The third chapter proposes the same project under a new light: the creation of a bootable SD card and the running of the project from within the SD card. The last chapter deals with the creation of a PS application so as to read a register. At the end of each project there is also a debugging session utilizing the J-TAG interface.

Contents

1	Petalinux	7
1	Installation procedure	7
1.1	i386 architecture	7
1.2	Petalinux Build Host packages	7
1.3	Petalinux SDK installation	8
2	Project 1: NAND gate - bare metal approach	9
1	Vivado project	9
1.1	Project creation	9
1.2	Behavioral description	10
1.3	Physical constraints	11
1.4	Design generation	11
1.5	FPGA programming	11
3	Project 2: NAND gate	13
1	Vivado project	13
2	Petalinux project	14
2.1	Base configuration	14
2.2	rootfs configuration	15
2.3	Booting the board	16
3	Programming the FPGA	17
4	J-TAG debugging	18
4	Project 3: blinking LED	19
1	Vivado project	19
1.1	Block configuration	21
2	Petalinux project	21
3	J-TAG debugging	22
5	Project 4: blinking LED with memory access	23
1	Vivado project	23
2	Code implementation	24
2.1	Driver	24
2.2	C program	24
3	Board booting	24
4	J-TAG debugging	24

Chapter 1

Petalinux

Petalinux is an embedded Software Development Kit - SDK - targeting FPGA-based System on a Chip - SoC - design or FPGA designs.

1 Installation procedure

Before the actual installation of Petalinux SDK, the creation of a compatible environment is requested. There are few steps to be followed in order to configure such environment:

- Adding the 32-bit architecture to the system
- Installing all the required packages for the Petalinux Build Host

1.1 i386 architecture

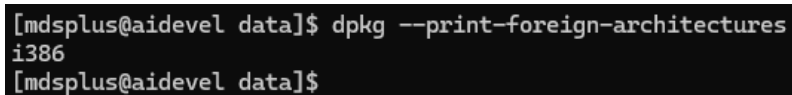
The 32bit architecture is essential for Petalinux to be installed. To add such architecture to the system, open the command prompt and type the following command:

```
$ sudo dpkg --add-architecture i386
```

It is now possible to check the availability of the architecture by typing

```
$ dpkg --print-foreign-architectures
```

and an output similar to that of Figure 1.1 will appear.

A terminal window with a black background and white text. The prompt is [mdsplus@aidevel data]\$. The command dpkg --print-foreign-architectures is entered, and the output is i386. The prompt returns to [mdsplus@aidevel data]\$.

```
[mdsplus@aidevel data]$ dpkg --print-foreign-architectures
i386
[mdsplus@aidevel data]$
```

Figure 1.1: Successful installation of i386 architecture.

1.2 Petalinux Build Host packages

From this page, download the file at the bottom. Create the folder in which petalinux will be installed with the command

```
mkdir <folder_name>
```

and copy the file from the download folder to the this final folder. Within this example, such folder is named ptlnx. To start the installation of all the packages type

```
chmod +x <installer_file_name>
sudo ./<file_name>
```

Once the installation is complete, all the packages should have been installed.

1.3 Petalinux SDK installation

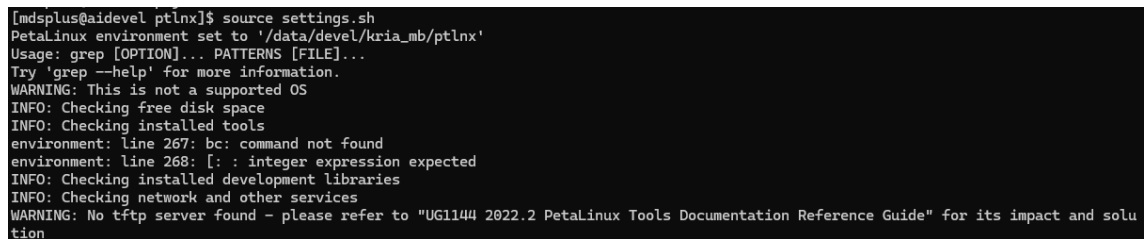
Now it is possible to begin the actual installation of Petalinux. From the AMD download page, download the needed version and move it to the installation folder. From this page also download the .BPS file relative to the board to be used. This latter file has to be moved into the folder in which the petalinux project are saved. Now type

```
chmod +x <installer_file_name>
./<file_name>
```

Now the installation should begin. By following the instruction, the installation process should proceed without any inconvenience, even though it might take a while. Finally, to set the Petalinux environment into the directory in which it is installed, type

```
source settings.sh
```

from which the shell output is highlighted in Fig. 1.2 is shown. Now the system should be all set.



```
[mdsplus@aidevel ptlnx]$ source settings.sh
Petalinux environment set to '/data/devel/kria_mb/ptlnx'
Usage: grep [OPTION]... PATTERNS [FILE]...
Try 'grep --help' for more information.
WARNING: This is not a supported OS
INFO: Checking free disk space
INFO: Checking installed tools
environment: line 267: bc: command not found
environment: line 268: [: : integer expression expected
INFO: Checking installed development libraries
INFO: Checking network and other services
WARNING: No tftp server found - please refer to "UG1144 2022.2 PetaLinux Tools Documentation Reference Guide" for its impact and solution
```

Figure 1.2: Setting petalinux environment.

Chapter 2

Project 1: NAND gate - bare metal approach

Within this chapter it will be explained how to program a FPGA, in this specific case the one mounted on the KR260 starter kit, with the so-called bare metal approach.

1 Vivado project

Vivado is a software developed by AMD which is oriented to programming Field Programmable Logic Arrays - FPGAs - which supports various board and modules. It is a free software, even though an account is requested for the download(s).

The example project that will be discussed within this chapter is meant only to give an introduction to the Vivado-FPGA programming world, which is way more vast and deeper. The description is quite simple: a simple NAND gate which controls a user defined LED mounted on the development board. As it can be seen the implementation is quite trivial,

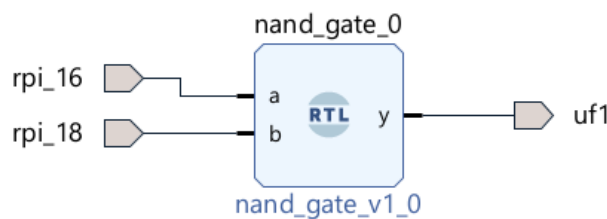


Figure 2.1: NAND gate Vivado block design.

consisting only of one module, which is the behavioral description of the NAND gate.

1.1 Project creation

Once Vivado is open and the start page loaded, create a new project named `nand_bm` (or whatever other name) and skip directly to the board selection window. In this window, select the name of the development board it is about to be used and create the project.

The first two tasks to carry out are NAND gate behavioral description file creation and the physical constraint constraints definition.

1.2 Behavioral description

Move to **Sources** → **Design Sources** → right-click on this latter one → **Add Sources**. A window like the one shown in Figure 2.2 should appear. Select **Add or create design**

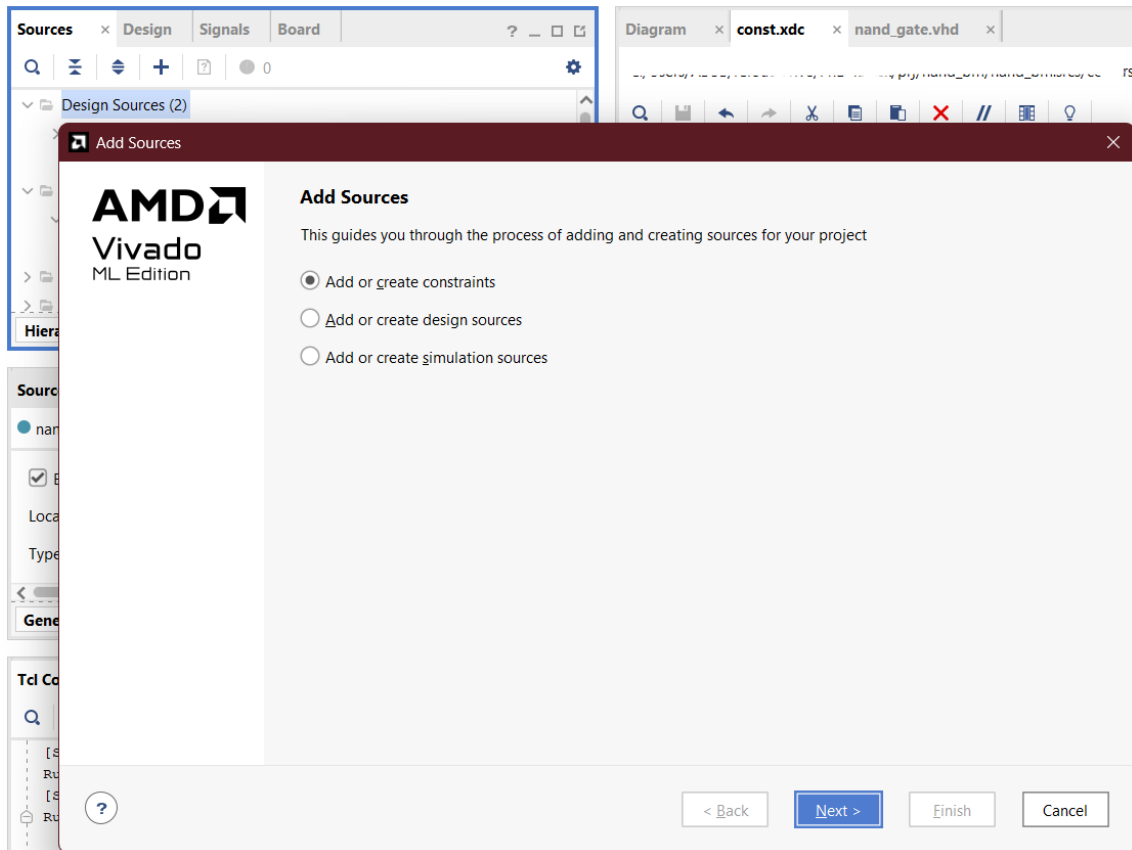


Figure 2.2: Add Sources window.

sources → **Next** → **Create file** → enter the file name (nand_gate) → **Finish**. Once it is done, a window to configure the inputs and the outputs of the gate should open. Such procedure can be skipped, providing that those information are defined in the description. The code to write down is the following:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity nand_gate is
    Port ( a, b : in STD_LOGIC;
           y : out STD_LOGIC);
end nand_gate;

architecture Behavioral of nand_gate is
begin
    y <= a nand b;
```

end Behavioral;

The project now has to be saved.

1.3 Physical constraints

The physical constraint file is the file that dictate which pin is connect to which within the development board and it of crucial importance for a real life application. Following a procedure similar to the one described for the behavioral description (**Sources** → **Constraints** → right-click on this latter one → **Add Sources** → **Add or create constraints** → **Next** → **Create file** → enter the file name (const) → **Finish**), the code to be written down is:

```
set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]
```

```
#User defined pins
```

```
set_property PACKAGE_PIN F8 [get_ports {uf1}]
```

```
set_property IOSTANDARD LVCMOS18 [get_ports {uf1}]
```

```
set_property PACKAGE_PIN AA12 [get_ports {rpi_16}]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports {rpi_16}]
```

```
set_property PACKAGE_PIN Y9 [get_ports {rpi_18}]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports {rpi_18}]
```

A bit of description it is needed to understand what those few lines of code do. The first line dictate that the bitstream that will be generated for this project will be compressed. The other lines define the port at which the buttons and the LED are connected (those information are provided from the datasheet of the board). For example, the pin at which the LED is connected is defined as a **standard I/O** with a voltage logic of **1.8V (LVCMOS18)**. In a similar way, the other two pins have the same characteristics, but with a voltage logic of **3.3V (LVCMOS33)**.

1.4 Design generation

Under the section of **IP INTEGRATOR**, click on **Create Block Design**: a new window will open. Move back to **Design Sources** → right click on the description file created earlier → **Add Module to Block Design**. To create a physical connection to the board pins, move the mouse tip on the block pins and type **Ctrl+T** to add a connection. By clicking on the created pins, a window named **External Port Properties** should appear. Rename those pins with the name define in the constraints file. Now the design should look like the one of Figure 2.1

1.5 FPGA programming

To program the FPGA is needed a file that can be compared to the .hex file that will be generated for the programming of the a microcontroller (ATmega, STM32, ...). Move to the **PROGRAM AND DEBUG** section and click on **Generate Bitstream**. Follow the procedure and let the bitstream to be generated. The last step is to effectively program the device. Open the **Open Hardware Manager** and click on **Open Target** → **Auto Connect**, then click on **Program Device**.

Chapter 3

Project 2: NAND gate

Once one grasp the basic procedure to program an FPGA, by natural extension it should be asked how it is possible to exploit the full power of the SoM. This chapter offers a good start point, but it is just the tip of the iceberg.

1 Vivado project

The project itself isn't changed: there is always the NAND gate. The only thing that is further added is the ability to boot the KR260 starter kit and to connect to it through the J-TAG interface. However, easy as it may seem, the procedure to follow is quite delicate and there are a lot of things to be taken care of. The project will further analyze two ways of programming the FPGA: a) from within the SoM OS and b) at the boot of the board. Apart from the NAND design blokk, the project has been created from an

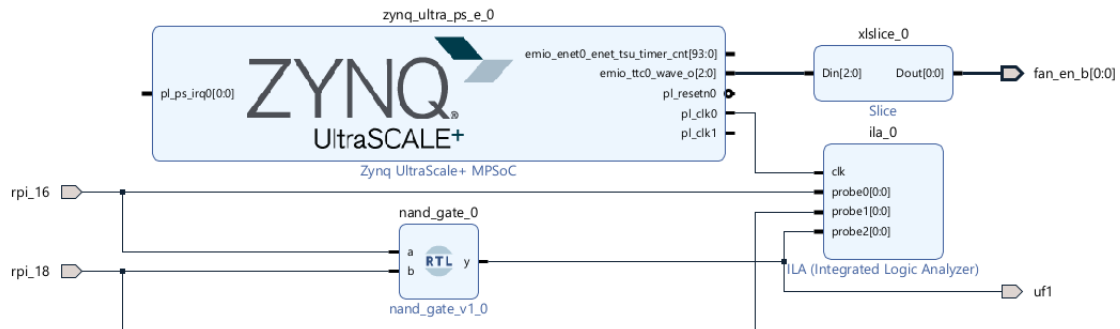


Figure 3.1: NAND gate Vivado block design.

example project given by AMD (this will include both the Zynq UltraScale+ MPSoC and the Slice). The ILA (Integrated Logic Analyzer) block serves the purpose to allow the view of the interesting waveform of the implementation during the debugging.

To create the project, open the Vivado main page, click on **Open Example Project** → **Next** → **Kria Starter Kit Design Example** → select the KR260 board → rename the project → **Default Bitstream** → create the project. To include both the source code of the NAND, follow the procedure explained in the previous chapter. The constrain file

already exist, but it needs to be modified, adding the lines of code of the precious file. Once it is done, the file should look like in the following.

```
set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]
```

```
#Fan Speed Enable
```

```
set_property PACKAGE_PIN A12 [get_ports {fan_en_b}]
set_property IOSTANDARD LVCMOS33 [get_ports {fan_en_b}]
set_property SLEW SLOW [get_ports {fan_en_b}]
set_property DRIVE 4 [get_ports {fan_en_b}]
```

```
#User defined pins
```

```
set_property PACKAGE_PIN F8 [get_ports {uf1}]
set_property IOSTANDARD LVCMOS18 [get_ports {uf1}]
```

```
set_property PACKAGE_PIN AA12 [get_ports {rpi_16}]
set_property IOSTANDARD LVCMOS33 [get_ports {rpi_16}]
```

```
set_property PACKAGE_PIN Y9 [get_ports {rpi_18}]
set_property IOSTANDARD LVCMOS33 [get_ports {rpi_18}]
```

Open the design under the **IP INTEGRATOR** section, add the NAND and the ILA blocks and connect them as it is showed in Figure 3.1. Run both **Connection Automation** and **Block Automation** and let Vivado manage all the other connection. If needed, remake the wanted connections. Now, following the same procedure defined earlier, connect the I/O pins to the physical pins. The project should now be ready. To generate the wrapper, save the design and click on **Create HDL wrapper**. Make the new wrapper the top file (right click on the wrapper just created → **Make Top File**), generate the bistream, then move to **File** → **Export** → **Export Hardware** → **Include Bitstream**. The project is all set and it is now possible to focus on the Petalinux part.

2 Petalinux project

From within the Petalinux installation folder, type

```
source ./settings.sh
```

so as to set the Petalinux environment. Create now the project with the command

```
petalinux-create --type project -s <path_to_bsp_file> --name <project_name>
```

and once it is done, move into the folder just created. The following two steps are quite important, so it is recommended a bit more of attention.

2.1 Base configuration

The first configuration procedure aims to configure the core of the OS, i.e. to which architecture is addressed and which features are to be enabled. With the command

```
petalinux-config --get-hw-description <path_to_xsa_file>
```

a menu window will open (see Figure 3.2), from which, moving into the various directories, the following packages must be enabled (or disabled):

- **FPGA Manager** → **[*]FPGA Manager**
- **Image Packing Configuration** → **[*]Copy final images to tftpboot**

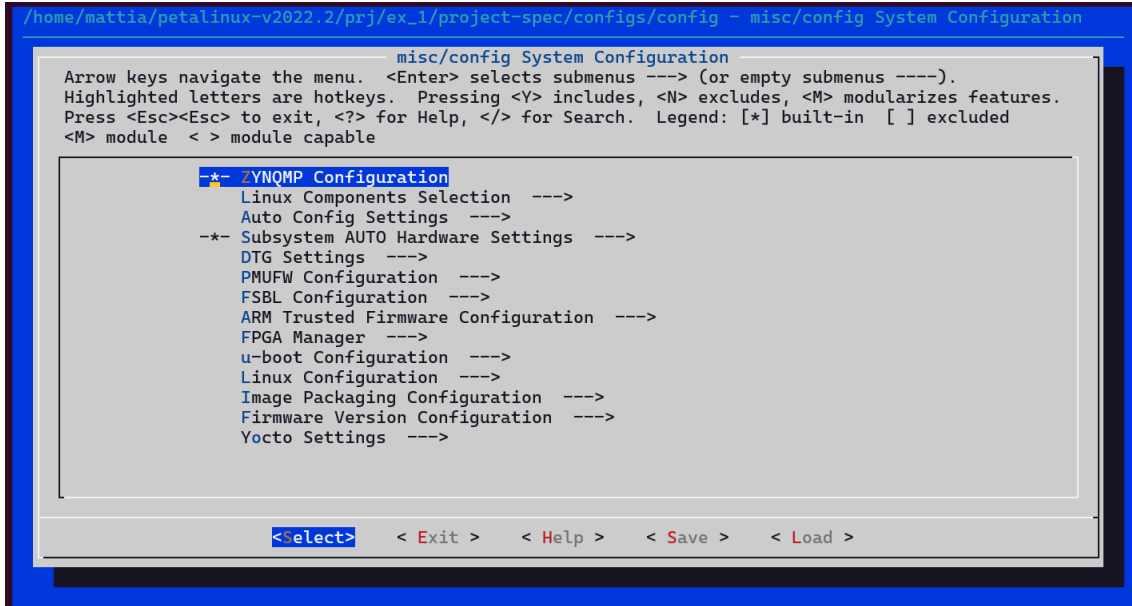


Figure 3.2: Base configuration window.

The FPGA manager will allow the FPGA to be programmed even during the operation of the board with few commands. However, it will generate an warning message during the creation of the BOOT.bin file.

2.2 rootfs configuration

The system to be created, in order to work, has to have some packages included, such as **bootget**, **gcc**, **XRT** and acceleration. The installation menu will open with the command

```
petalinux-config -c rootfs
```

from which the following packages have to be installed:

- **Filesystem Packages** → **console** → **utils** → **git** → **[*]git**
- **Filesystem Packages** → **base** → **dnf** → **[*]dnf**
- **Filesystem Packages** → **x11** → **base** → **libdrm** → **[*]libdrm**, **[*]libdrm-tests**, **[*]libdrm-kms**
- **Filesystem Packages** → **libs** → **xrt** → **[*]xrt**, **[*]xrt-dev**
- **Filesystem Packages** → **libs** → **zocl** → **[*]zocl**
- **Filesystem Packages** → **libs** → **opencl-headers** → **[*]opencl-headers**
- **Filesystem Packages** → **libs** → **opencl-clhpp** → **[*]opencl-clhpp**, **[*]opencl-clhpp-dev**
- **Filesystem Packages** → **bootgen** → **[*]bootgen**, **[*]bootgen-dev**

-
- **Filesystem Packages** → misc → packagegroup-core-buildessential → [*]packagegroup-core-buildessential, [*]packagegroup-core-buildessential-dev
 - **Filesystem Packages** → misc → gcc-runtime → [*]libstdc++, [*]libstdc++-dev
 - **Filesystem Packages** → misc → gcr → [*]gcr, [*]gcr-dev
 - **Filesystem Packages** → misc → gdb → [*]gdb, [*]gdb-dev
 - **Filesystem Packages** → misc → glib-2.0 → [*]glib-2.0, [*]glib-2.0-dev
 - **Filesystem Packages** → misc → glibc → [*]glibc, [*]glibc-dev, [*]ldd
 - **Petalinux Package Groups** → packagegroup-petalinux → [*]packagegroup-petalinux
 - **Petalinux Package Groups** → packagegroup-petalinux-gstreamer → [*]packagegroup-petalinux-gstreamer
 - **Petalinux Package Groups** → packagegroup-petalinux-opencv → [*]packagegroup-petalinux-opencv
 - **Petalinux Package Groups** → packagegroup-petalinux-v4lutils → [*]packagegroup-petalinux-v4lutils
 - **Petalinux Package Groups** → packagegroup-petalinux-x11 → [*]packagegroup-petalinux-x11

After this boring procedure, it is time to build the project with the command

```
petalinux-build
```

In the case of building failure with error messages not related to the implementation, try to clear out the current files by typing

```
petalinux-config -x mr proper
```

command which is used to deep cleaning the workspace, similar to how it's used in the Linux kernel build system.

2.3 Booting the board

Since the board boots from a SD card, the system configuration just created has to be converted into a bootable image for the card. First step to carry out is the creation of the BOOT.bin file, which is of quite importance. To do so, after the project build, the command line is (on the same line)

```
petalinux-package --boot --fsbl <path_to_elf_file> --fpga <path_to_bitstream> --u-boot
```

in which the .elf file is usually located under the path images/linux/. The second step is to create a .wic image for the SD card to be flashed in. The command to do that is (on the same line)

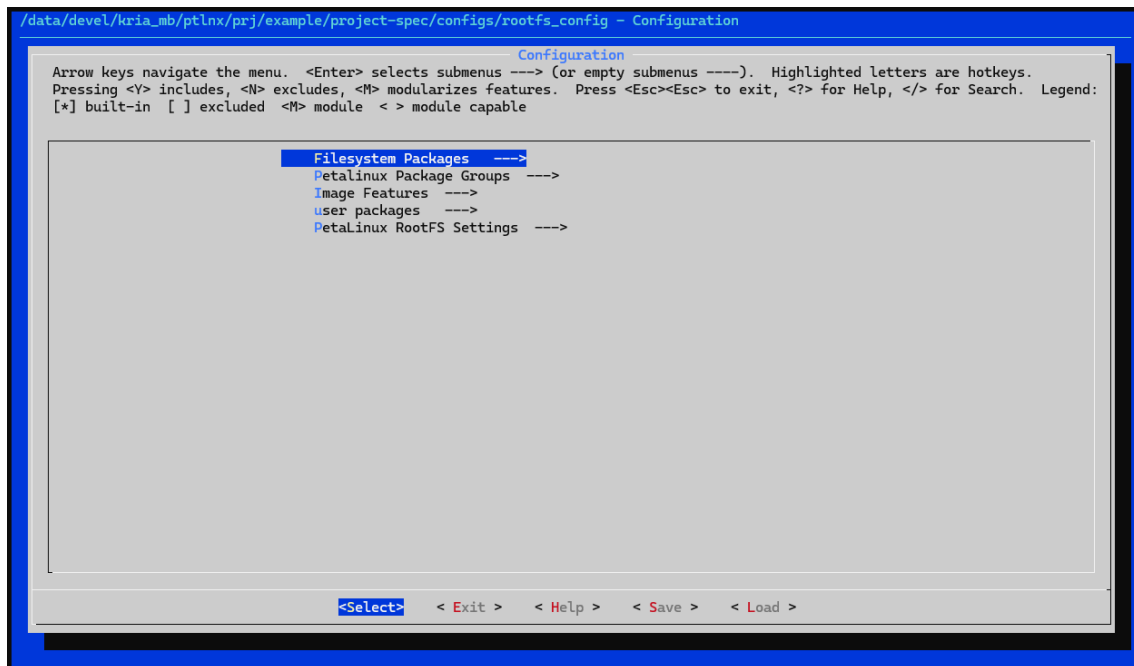


Figure 3.3: rootfs configuration window.

```
petalinux-package --wic --images-dir images/linux/ --bootfiles
"ramdisk.cpio.gz.u-boot,boot.scr,Image,system.dtb,system-zynqmp-sck-kr-g
-revB.dtb" --disk-name "sda"
```

from which the .wic image will be generated and found in the same folder as the .elf file. To flash the SD card make use of a third-part software, such as balenaEtcher or something similar. Once the flash is done, insert the SD in the board and power it up. With a third part software, such as PuTTY, connect to the serial port (Baud Rate 115200). The system will require to inset some credential and the following may be used:

```
login : petalinux
password : your_password
```

and now everything is set. To check if the packages are actually installed, type in the serial terminal

```
gcc --version
bootgen
```

and some messages relating to the version or how to use them will be displayed.

3 Programming the FPGA

So far, only the boot was done with the procedure defined. The FPGA is not actually programmed. To do so, two ways can be used: the On-Run programming and the Boot programming. The first one allows for more flexibility and has the advantage to let the FPGA to be programmed even when the system is running, without the need to repeat all the steps previously explained. The second, thought being more rigid, allows for the FPGA to be programmed each time the system boots up. Within this example, the first approach has been chosen. To actually program the FPGA in after the system is booted,

there is the need to convert the bitstream into a .bin file and then "manually" load it into the FPGA through the FPGA manager. Both .bit and the .bif file are to be copied in the SD card into an accessible folder. The .bif file has the following command lines:

```
all:
{
    bit_file_name.bit
}
```

and through that the .bin file can be created using the command

```
sudo bootgen -image <bif_file> -arch zynqmp -process_bitstream bin
```

Now the .bin file has to be copied into the folder at the /lib/firmware path. The last step to program the FPGA is to load the file with the command:

```
sudo sh -c 'echo "bin_file" > /sys/class/fpga_manager/fpga0/firmware'
```

4 J-TAG debugging

Vivado offers a very valid way to debug the program just created. This is made possible thanks to the ILA (Integrated Logic Analyzer), which "reads" the internal signals to which its probes are connected, save them to a RAM BUS and the, through J-TAG, they can be seen in a similar way an oscilloscope plots the red signals. Actually, its functioning it very similar to an oscilloscope itself.

Once the FPGA is programmed, open Vivado's **Hardware Manager** and repeat the same connecting procedure as for the bare programming. After the successful connection, in the Hardware window, under the FPGA's name, the ILA hw will appear. Double click on it and in the main window will be displayed all the selected signals alongside with their waveform. In the **Trigger Set Up** window, chose the triggering signal and triggering condition and then click on the Play button of the main window.

For this project, the debugging may be less interesting than effectively seeing the board's led turn on and off according to the NAND's truth table. However, knowing how to set up the debugging is quite useful, especially for future, more complex applications. In the following chapter a more debug-reliable example is explained.

Chapter 4

Project 3: blinking LED

This project differs from the previous one: in this chapter, the Processing System will be set up without the creation of an example project. It will add more time to the initial configuration, but it is an important step in order to understand what it has to be modified to enable different peripherals within the microprocessor.

1 Vivado project

After having created a new Vivado project with the same procedure defined in Chapter 2, the constraints file and the source file have to be created. The code is reported in the following:

- Constraints file:

```
set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]

#Fan Speed Enable
set_property PACKAGE_PIN A12 [get_ports {fan_en_b}]
set_property IOSTANDARD LVCMOS33 [get_ports {fan_en_b}]
set_property SLEW SLOW [get_ports {fan_en_b}]
set_property DRIVE 4 [get_ports {fan_en_b}]

#User defined pins
set_property PACKAGE_PIN F8 [get_ports {uf1}]
set_property IOSTANDARD LVCMOS18 [get_ports {uf1}]
```

- Source file #1:

```
-- Clock Divider
-- Every match, the output signal is toggled
-- The output frequency is given by: f_out = f_in / (2*max_val)

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
```

```
entity prescaler is
```

```
    Port ( clk_in, reset    : in STD_LOGIC;
           clk_out          : out STD_LOGIC);
```

```
end prescaler;
```

```
architecture Behavioral of prescaler is
```

```
    constant    max_val : integer                := 500000;
    signal       counter : integer range 0 to max_val-1 := 0;
    signal       tmp      : std_logic              := '1';
```

```
begin
```

```
    process(clk_in, reset)
    begin
        if reset = '0'
        then
            tmp <= '0';
            counter <= 0;
        else
            if rising_edge(clk_in)
            then
                if counter = max_val-1 -- Counts from 0 to max_val-1
                then
                    counter <= 0;
                    tmp <= not tmp;
                else
                    counter <= counter + 1;
                end if;
            end if;
        end if;
    end process;
```

```
    clk_out <= tmp;
```

```
end Behavioral;
```

- Source file #2 (AND gate to be inserted after the clocking wizard):

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity and_gate is
```

```
    Port ( a, b : in STD_LOGIC;
           y      : out STD_LOGIC);
```


3 J-TAG debugging

The main difference between this project and the previous one is the presence of a clock source that will allow for the ILA to trigger and synchronize the displayed signals. After having programmed the FPGA, opened the **Hardware Manager** and connected the device, Vivado should automatically find the debug-probes file (if not, just re-programm the FPGA). Between the various signals, chose one to be referred as trigger and define the trigger event (equal to a value, rising or falling edge...) and run the trigger event for the ILA core.

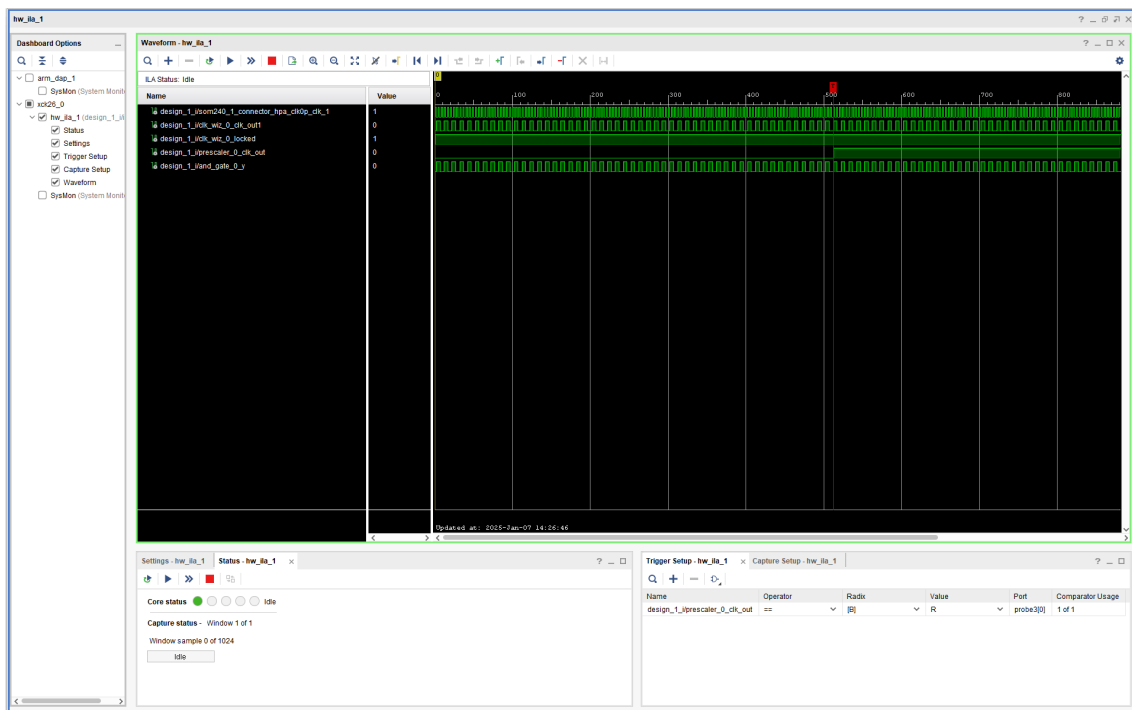


Figure 4.2: J-Tag debugging with ILA core (triggered on the blinking signal).

Project 4: blinking LED with memory access

1 Vivado project

[illegible]

application was set to be 128bit (there is no need for higher space, since only a bit is needed to be read and written) and the memory address to `0xB0000000`. The component used to perform R/W operations is named *axi_cfg_register*. As always, after having generated the bitstream, it is needed to export the hardware including the bitstream with it. Also the creation of the Petalinux project is not different from the previous. What changes is the procedure after the boot of the board.

Name	Interface	Slave Segment	Master Base Address	Range	Master High Address
Network 0					
/zynq_ultra_ps_e_0					
/zynq_ultra_ps_e_0/Data (40 address bits : 0x00A0000000 [256M], 0x0400000000 [4G], 0x1000000000 [224G], 0x00B0000000 [256M], 0x0500000000 [4G])					
/axi_cfg_register_0/s_axi	s_axi	reg0	0xB000_0000	128	0xB000_007F

Figure 5.2: Address editor.

2 Code implementation

After having successfully created and built the project, to perform R/W operations on the memory allocated by the `cfg_register`, there is the necessity to define a) the driver and b) the program with which the operations are performed.

2.1 Driver

The generation of the driver is beyond the scope of this guide. It is assumed that the corresponding `.c` and `.h` files are already available. In this discussion it is explained how to create a driver needed to blink the led, but the same procedure can be applied to a wide variety of applications. Now, assuming the `.c` and `.h` files are respectively named

```
driver_name.c
driver_name.h
```

then, the creation of the driver modules is performed with the command

```
petalinux-create -t modules --name driver_name --enable
```

after which

```
petalinux-build -c driver_name
```

will build the module. Now, the aforementioned files `.c` and `.h` are to be copied into the `project-spec/meta-user/recipes-modules/driver_name/files/` path. Finally, to build the module

```
petalinux-build -c driver_name
```

2.2 C program

3 Board booting

4 J-TAG debugging